*Last updated on **Mar 31, 2026***

Tutorial: Create and manage Delta Lake tables

This tutorial demonstrates common Delta table operations using sample data. Delta Lake is the optimized storage layer that provides the foundation for tables on Databricks. Unless otherwise specified, all tables on Databricks are Delta tables.

Before you begin

To complete this tutorial, you need:

- Permission to use an existing compute resource or create a new compute resource. See Compute.
- Unity Catalog permissions: `USE CATALOG`, `USE SCHEMA`, and `CREATE TABLE` on the `workspace` catalog. To set these permissions, see your Databricks administrator or Unity Catalog privileges reference.

These examples rely on a dataset called *Synthetic Person Records: 10K to 10M Records*. This dataset contains fictitious records of people, including their first and last names, gender, and age.

First, download the dataset for this tutorial.

1. Visit the Synthetic Person Records: 10K to 10M Records page on Kaggle.
2. Click **Download** and then **Download dataset as zip**. This downloads a file named `archive.zip` to your local machine.
3. Extract the `archive` folder from the `archive.zip` file.

Next, upload the `person_10000.csv` dataset to a Unity Catalog volume within your Databricks workspace. Databricks recommends uploading your data to a Unity Catalog volume because volumes provide capabilities for accessing, storing, governing, and organizing files.

1. Open Catalog Explorer by clicking **Catalog** in the sidebar.

2. In Catalog Explorer, click + **Add data** and **Create a volume**.
3. Name the volume `my-volume` and select **Managed volume** as the volume type.
4. Select the `workspace` catalog and the `default` schema, and then click **Create**.
5. Open `my-volume` and click **Upload to this volume**.
6. Drag and drop or browse to and select the `person_10000.csv` file from within the `archive` folder on your local machine.
7. Click **Upload**.

Last, create a notebook for running the sample code.

1. Click + **New** in the sidebar.
2. Click **Notebook** to create a new notebook.
3. Choose a language for the notebook.

Create a table

Create a new Unity Catalog managed table named `workspace.default.people_10k` from `person_10000.csv`. Delta Lake is the default for all table creation, read, and write commands in Databricks.

Python Scala SQL

Python

```
from pyspark.sql.types import StructType, StructField, IntegerType, StringType

schema = StructType([
  StructField("id", IntegerType(), True),
  StructField("firstName", StringType(), True),
  StructField("lastName", StringType(), True),
  StructField("gender", StringType(), True),
  StructField("age", IntegerType(), True)
])

df = spark.read.format("csv").option("header",
True).schema(schema).load("/Volumes/workspace/default/my-
volume/person_10000.csv")

# Create the table if it does not exist. Otherwise, replace the existing table.
df.writeTo("workspace.default.people_10k").createOrReplace()
```

Ask Genie

```
# If you know the table does not already exist, you can use this command
instead.
# df.write.saveAsTable("workspace.default.people_10k")

# View the new table.
df = spark.read.table("workspace.default.people_10k")
display(df)
```

There are several different ways to create or clone tables. For more information, see CREATE TABLE.

In Databricks Runtime 13.3 LTS and above, you can use CREATE TABLE LIKE to create a new empty Delta table that duplicates the schema and table properties of a source Delta table. This can be useful when promoting tables from a development environment into production.

SQL

```
CREATE TABLE workspace.default.people_10k_prod LIKE
workspace.default.people_10k
```

> **PREVIEW**
>
> This feature is in Public Preview.

Use the `DeltaTableBuilder` API for Python and Scala to create an empty table. Compared to `DataFrameWriter` and `DataFrameWriterV2`, the `DeltaTableBuilder` API makes it easier to specify additional information like column comments, table properties, and generated columns.

Python Scala

Python

```
from delta.tables import DeltaTable

(
  DeltaTable.createIfNotExists(spark)
    .tableName("workspace.default.people_10k_2")
    .addColumn("id", "INT")
    .addColumn("firstName", "STRING")
    .addColumn("lastName", "STRING", comment="surname")
    .addColumn("gender", "STRING")
```

Ask Genie

```
    .addColumn("age", "INT")
    .execute()
)

display(spark.read.table("workspace.default.people_10k_2"))
```

Upsert to a table

Modify existing records in a table or add new ones using an operation called *upsert*. To merge a set of updates and insertions into an existing Delta table, use the `DeltaTable.merge` method in Python and Scala and the MERGE INTO statement in SQL.

For example, merge data from the source table `people_10k_updates` to the target Delta table `workspace.default.people_10k`. When there is a matching row in both tables, Delta Lake updates the data column using the given expression. When there is no matching row, Delta Lake adds a new row.

Python **Scala** **SQL**

Python

```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType
from delta.tables import DeltaTable

schema = StructType([
  StructField("id", IntegerType(), True),
  StructField("firstName", StringType(), True),
  StructField("lastName", StringType(), True),
  StructField("gender", StringType(), True),
  StructField("age", IntegerType(), True)
])

data = [
  (10001, 'Billy', 'Luppitt', 'M', 55),
  (10002, 'Mary', 'Smith', 'F', 98),
  (10003, 'Elias', 'Leadbetter', 'M', 48),
  (10004, 'Jane', 'Doe', 'F', 30),
  (10005, 'Joshua', '', 'M', 90),
  (10006, 'Ginger', '', 'F', 16),
]

# Create the source table if it does not exist. Otherwise, replace the existing
source table.
people_10k_updates = spark.createDataFrame(data, schema)
```

```
people_10k_updates.createOrReplaceTempView("people_10k_updates")

# Merge the source and target tables.
deltaTable = DeltaTable.forName(spark, 'workspace.default.people_10k')

(deltaTable.alias("people_10k")
  .merge(
    people_10k_updates.alias("people_10k_updates"),
    "people_10k.id = people_10k_updates.id")
  .whenMatchedUpdateAll()
  .whenNotMatchedInsertAll()
  .execute()
)

# View the additions to the table.
df = spark.read.table("workspace.default.people_10k")
df_filtered = df.filter(df["id"] >= 10001)
display(df_filtered)
```

In SQL, the `*` operator updates or inserts all columns in the target table, assuming that the source table has the same columns as the target table. If the target table does not have the same columns, the query throws an analysis error. Also, you must specify a value for every column in your table when you perform an insert operation. The column values can be empty, for example, `''`. When you perform an insert operation, you do not need to update all values.

Read a table

Use the table name or path to access data in Delta tables. To access Unity Catalog managed tables, use a fully-qualified table name. Path-based access is only supported for volumes and external tables, not for managed tables. For more information, see Path rules and access in Unity Catalog volumes.

Python Scala SQL

Python

```
people_df = spark.read.table("workspace.default.people_10k")
display(people_df)
```

Ask Genie

Write to a table

Delta Lake uses the standard syntax for writing data to tables. To add new data to an existing Delta table, use the append mode. Unlike upserting, writing to a table does not check for duplicate records.

Python Scala SQL

Python

```python
from pyspark.sql.types import StructType, StructField, StringType, IntegerType
from pyspark.sql.functions import col

schema = StructType([
  StructField("id", IntegerType(), True),
  StructField("firstName", StringType(), True),
  StructField("lastName", StringType(), True),
  StructField("gender", StringType(), True),
  StructField("age", IntegerType(), True)
])

data = [
  (10007, 'Miku', 'Hatsune', 'F', 25)
]

# Create the new data.
df  = spark.createDataFrame(data, schema)

# Append the new data to the target table.
df.write.mode("append").saveAsTable("workspace.default.people_10k")

# View the new addition.
df = spark.read.table("workspace.default.people_10k")
df_filtered = df.filter(df["id"] == 10007)
display(df_filtered)
```

Databricks notebook cell outputs display a maximum of 10,000 rows or 2 MB, whichever is lower. Because `workspace.default.people_10k` contains more than 10,000 rows, only the first 10,000 rows appear in the notebook output for `display(df)`. The additional rows are present in the table, but are not rendered in the notebook output due to this limit. You can view the additional rows by specifically filtering for them.

To replace all the data in a table, use the overwrite mode.

Ask Genie

Python Scala SQL

Python

```
df.write.mode("overwrite").saveAsTable("workspace.default.people_10k")
```

Update a table

Update data in a Delta table based on a predicate. For example, change the values in the `gender` column from `Female` to `F`, from `Male` to `M`, and from `Other` to `O`.

Python Scala SQL

Python

```
from delta.tables import *
from pyspark.sql.functions import *

deltaTable = DeltaTable.forName(spark, "workspace.default.people_10k")

# Declare the predicate and update rows using a SQL-formatted string.
deltaTable.update(
  condition = "gender = 'Female'",
  set = { "gender": "'F'" }
)

# Declare the predicate and update rows using Spark SQL functions.
deltaTable.update(
  condition = col('gender') == 'Male',
  set = { 'gender': lit('M') }
)

deltaTable.update(
  condition = col('gender') == 'Other',
  set = { 'gender': lit('O') }
)

# View the updated table.
df = spark.read.table("workspace.default.people_10k")
display(df)
```

Delete from a table

Remove data that matches a predicate from a Delta table. For example, the code below demonstrates two delete operations: first deleting rows where age is less than 18, then deleting rows where age is less than 21.

Python Scala SQL

Python

```python
from delta.tables import *
from pyspark.sql.functions import *

deltaTable = DeltaTable.forName(spark, "workspace.default.people_10k")

# Declare the predicate and delete rows using a SQL-formatted string.
deltaTable.delete("age < '18'")

# Declare the predicate and delete rows using Spark SQL functions.
deltaTable.delete(col('age') < '21')

# View the updated table.
df = spark.read.table("workspace.default.people_10k")
display(df)
```

IMPORTANT

Deletion removes the data from the latest version of the Delta table but does not remove it from physical storage until the old versions are explicitly vacuumed. For more information, see vacuum.

Display table history

Use the `DeltaTable.history` method in Python and Scala and the DESCRIBE HISTORY statement in SQL to view the provenance information for each write to a table.

Python Scala SQL

Python

```
from delta.tables import *

deltaTable = DeltaTable.forName(spark, "workspace.default.people_10k")
display(deltaTable.history())
```

Query an earlier version of the table using time travel

Query an older snapshot of a Delta table using Delta Lake time travel. To query a specific version, use the table's version number or timestamp. For example, query version `0` or timestamp `2026-01-05T23:09:47.000+00:00` from the table's history.

Python Scala SQL

Python

```
from delta.tables import *

deltaTable = DeltaTable.forName(spark, "workspace.default.people_10k")
deltaHistory = deltaTable.history()

# Query using the version number.
display(deltaHistory.where("version == 0"))

# Query using the timestamp.
display(deltaHistory.where("timestamp == '2026-01-05T23:09:47.000+00:00'"))
```

For timestamps, only date or timestamp strings are accepted. For example, strings must be formatted as `"2026-01-05T22:43:15.000+00:00"` or `"2026-01-05 22:43:15"`.

Use `DataFrameReader` options to create a DataFrame from a Delta table that is fixed to a specific version or timestamp of the table.

Python Scala SQL

Python

```
# Query using the version number.
df = spark.read.option('versionAsOf', 0).table("workspace.default.people_10k")

# Query using the timestamp.
df = spark.read.option('timestampAsOf', '2026-01-
05T23:09:47.000+00:00').table("workspace.default.people_10k")

display(df)
```

For more information, see Work with table history.

Optimize a table

Multiple changes to a table can create several small files, which slows read query performance. Use the optimize operation to improve speed by combining small files into larger ones. See OPTIMIZE.

Python **Scala** **SQL**

Python

```
from delta.tables import *

deltaTable = DeltaTable.forName(spark, "workspace.default.people_10k")
deltaTable.optimize().executeCompaction()
```

> **NOTE**
>
> If predictive optimization is enabled, you do not need to optimize manually. Predictive optimization automatically manages maintenance tasks. For more information, see Predictive optimization for Unity Catalog managed tables.

Use liquid clustering

To further improve read performance, use liquid clustering to colocate related data. For example, enable clustering on the high cardinality column `firstName`. Use `OPTIMIZE FULL` to

apply clustering to all existing data. For more information, see Use liquid clustering for tables.

Python Scala SQL

Python

```
spark.sql("ALTER TABLE workspace.default.people_10k CLUSTER BY (firstName)")
spark.sql("OPTIMIZE FULL workspace.default.people_10k")
```

Clean up snapshots with the vacuum operation

Delta Lake has snapshot isolation for reads, which means that it is safe to run an optimize operation while other users or jobs are querying the table. However, you should eventually clean up old snapshots because doing so reduces storage costs, improves query performance, and ensures data compliance. Run the `VACUUM` operation to clean up old snapshots. See VACUUM.

Python Scala SQL

Python

```
from delta.tables import *

deltaTable = DeltaTable.forName(spark, "workspace.default.people_10k")
deltaTable.vacuum()
```

For more information on using the vacuum operation effectively, see Remove unused data files with vacuum.

Next steps

- Use liquid clustering for tables
- Best practices: Delta Lake

Ask Genie

Related resources

- What is Delta Lake in Databricks?

On this page

Was this page helpful?

Yes | No

Send feedback

Ask Genie